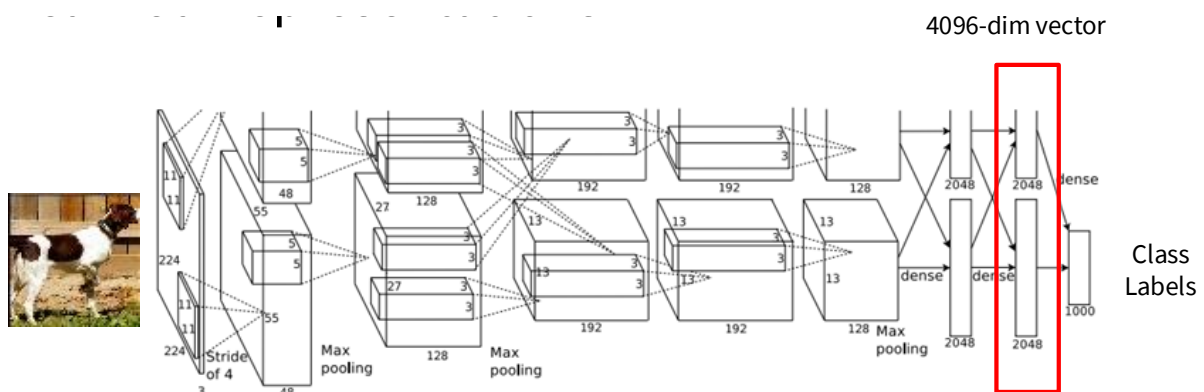


Self-Supervision

Luiz Velho
IMPA

Based on Slides from Stanford-CS231n-2025

Learned Representations

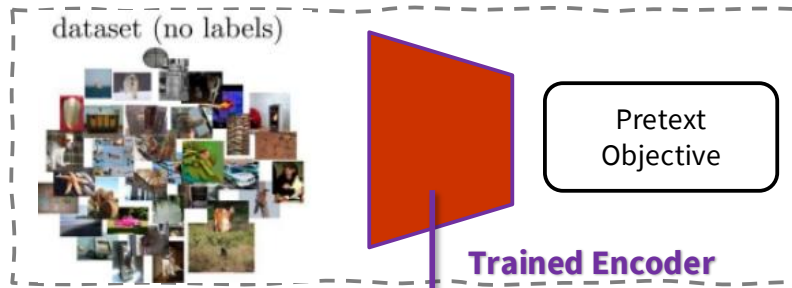


What is the problem with large-scale training?

- **We need a lot of labeled data**

Is there a way we can train neural networks without the need for huge manually labeled datasets?

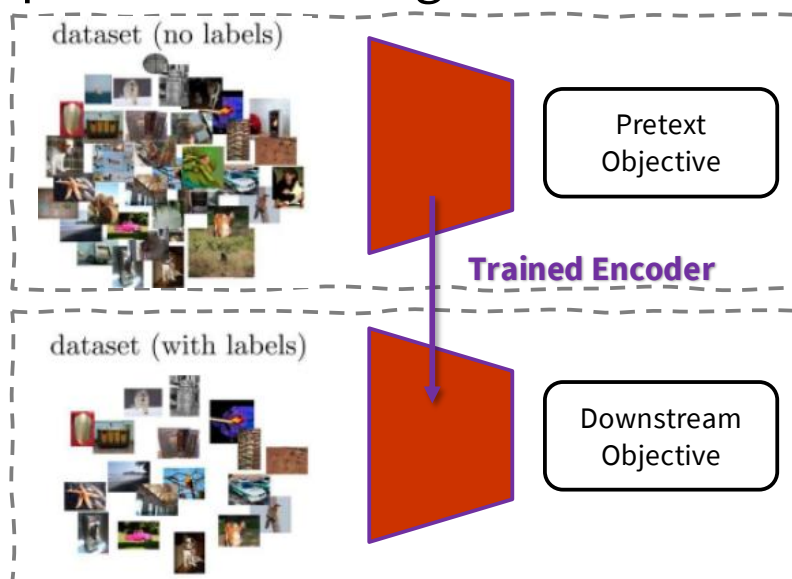
Self-Supervised Learning



Pretext Task

- Define a task based on the data itself
- No manual annotation
- Could be considered an **unsupervised** task;
- but we learn with supervised learning objectives, e.g., classification or regression.

Self-Supervised Learning



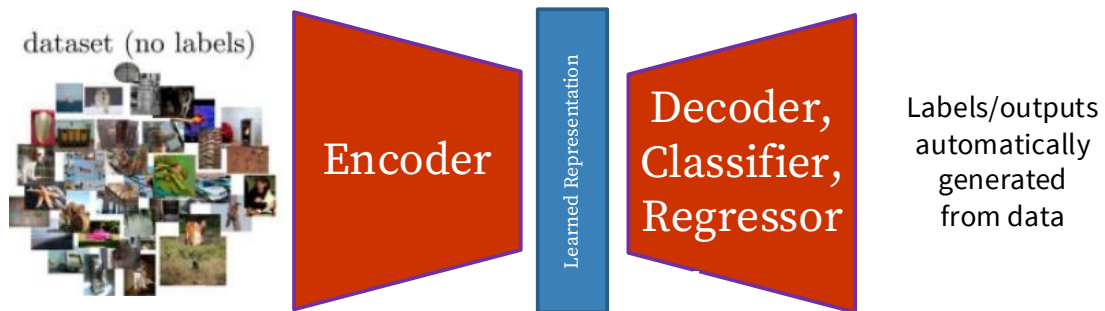
Pretext Task

- Define a task based on the data itself
- No manual annotation
- Could be considered an **unsupervised** task;
- but we learn with supervised learning objectives, e.g., classification or regression.

Downstream Task

- The application you care about
- You do not have large datasets
- The dataset is labeled

Self-Supervised Learning – Pretext Task



Self-supervised pretext tasks

Example: learn to predict image transformations / complete corrupted images

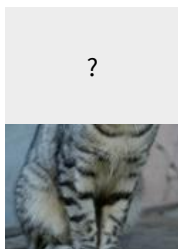
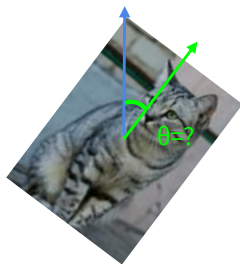
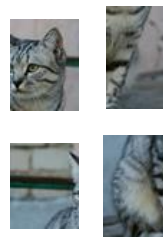


image completion



rotation prediction



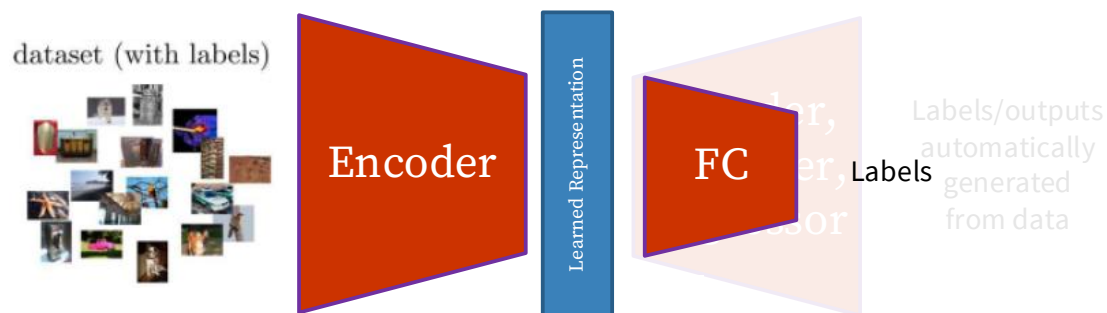
“jigsaw puzzle”



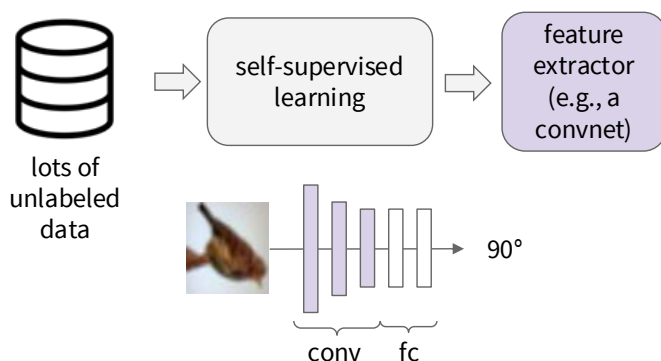
colorization

1. Solving the pretext tasks allow the model to learn good features.
2. We can automatically generate labels for the pretext tasks.

Self-Supervised Learning – Downstream Task

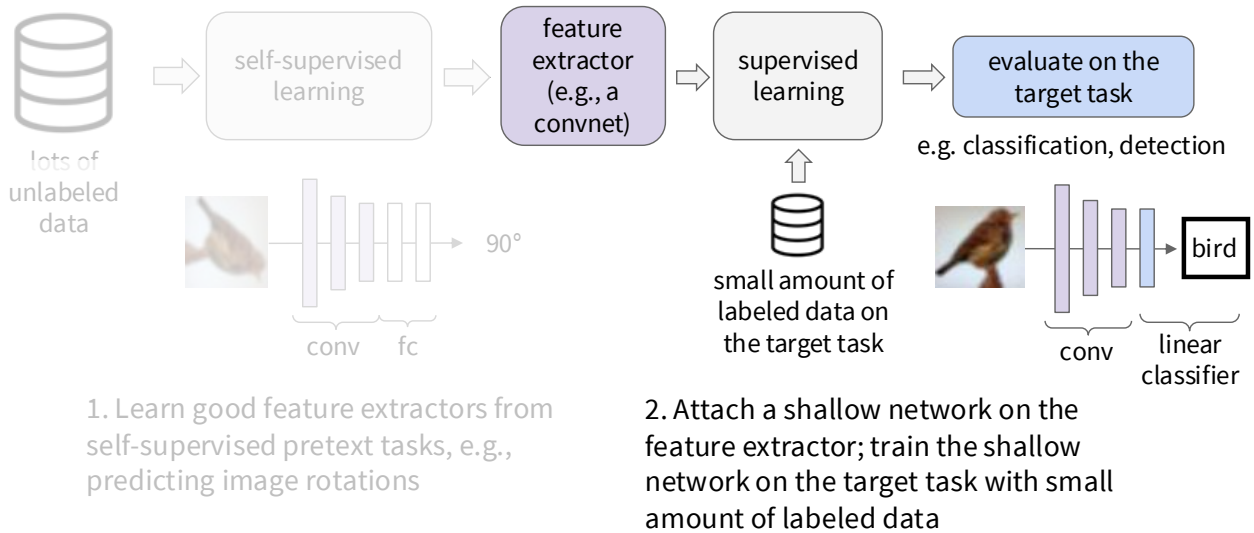


Example (1) Extracting Features



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

Example (2) Fine-Tuning with Labels



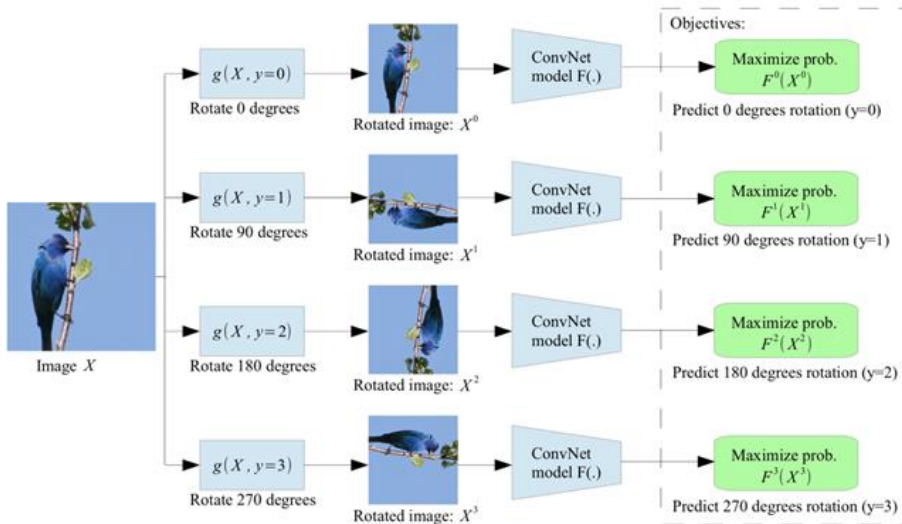
Pretext Tasks

Techniques

- Image Transformations
 - Rotation
 - Inpainting
 - Rearrangement
 - Coloring
- Reconstruction-Based
 - Masked Auto-Encoder

*Pretext Tasks
from Image Transformations*

Pretext task: predict rotations

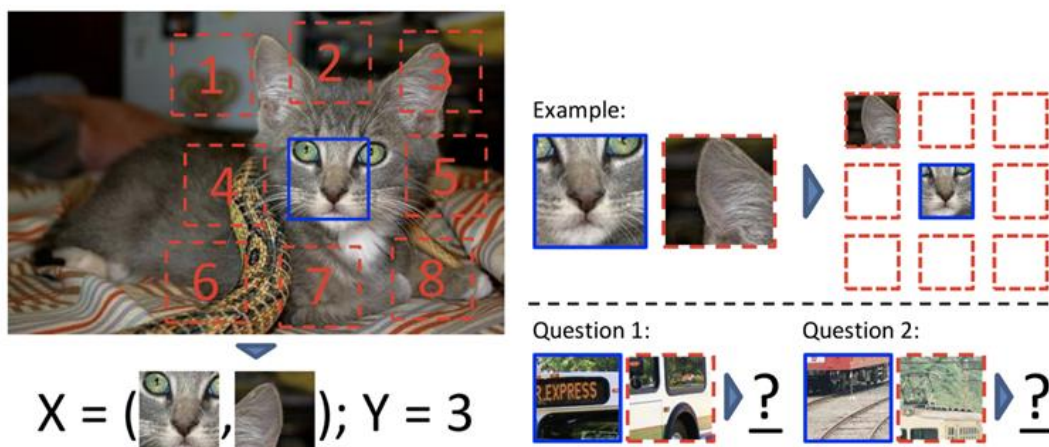


Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification)

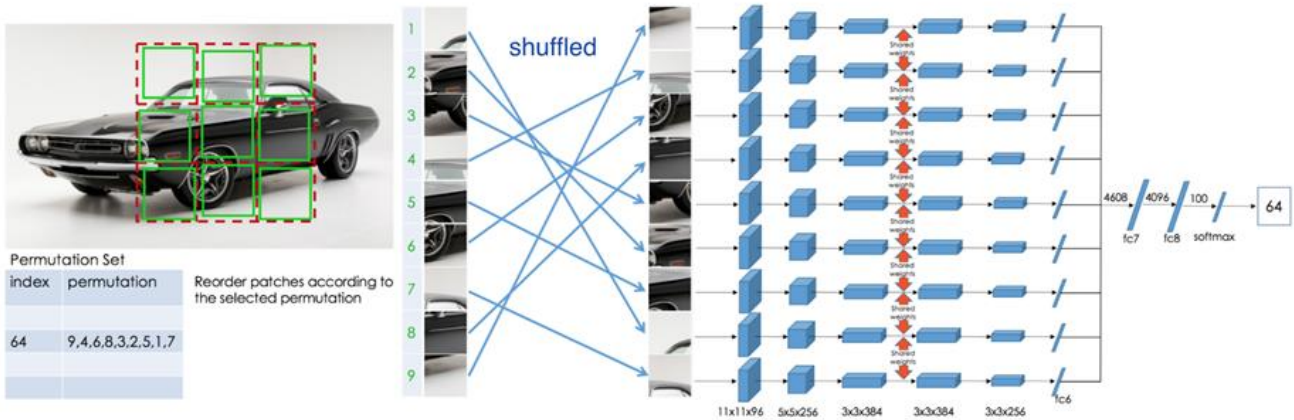
(Image source: [Gidaris et al. 2018](#))

Pretext task: predict relative patch locations



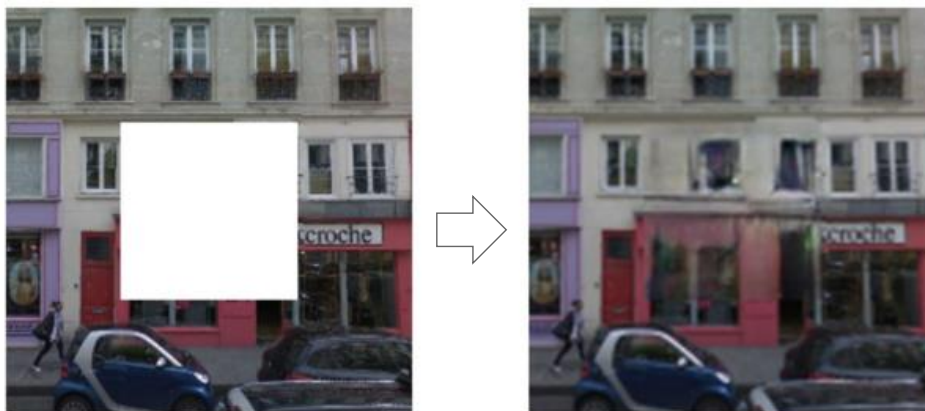
(Image source: [Doersch et al., 2015](#))

Pretext task: solving “jigsaw puzzles”



(Image source: [Noroozi & Favaro, 2016](#))

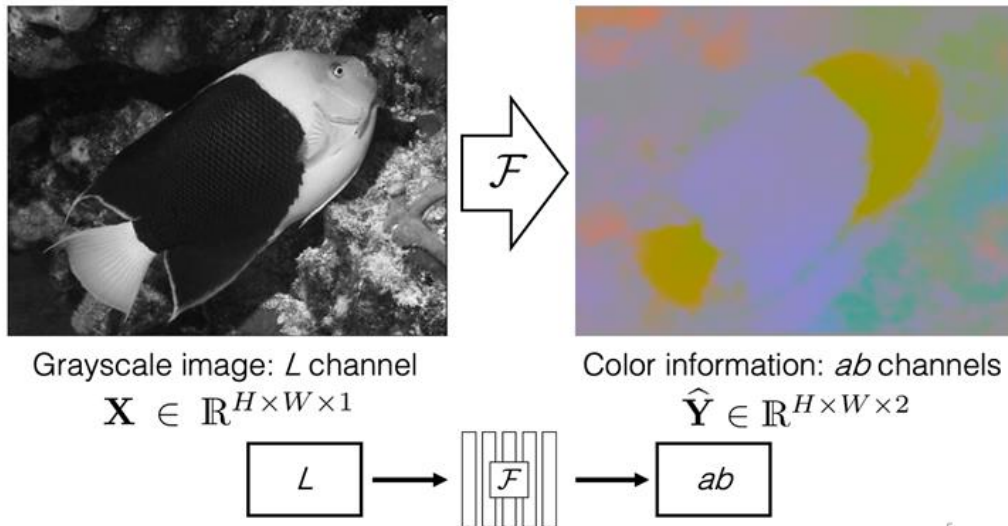
Pretext task: predict missing pixels (inpainting)



Context Encoders: Feature Learning by Inpainting (Pathak et al., 2016)

Source: [Pathak et al., 2016](#)

Pretext task: image coloring

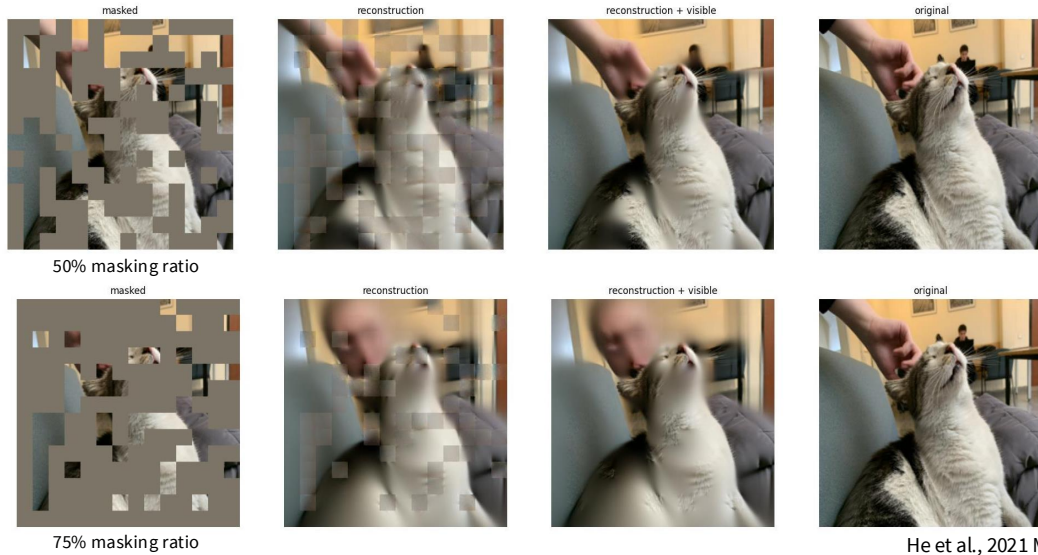


Source: Richard Zhang / Phillip Isola

Reconstruction-Based Learning

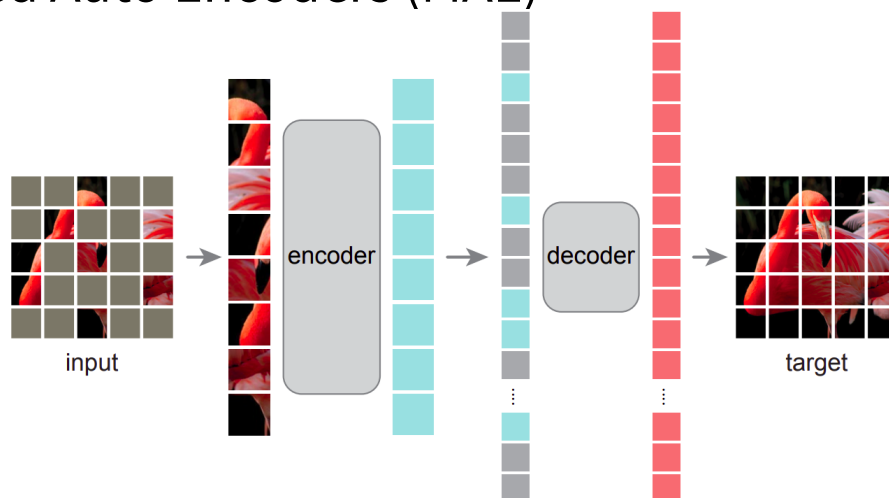
Masked Auto Encoders (MAE)

Reconstruction with a larger masked portion



He et al., 2021 Masked Autoencoders
Are Scalable Vision Learners

Masked Auto Encoders (MAE)



He et al., 2021 Masked Autoencoders
Are Scalable Vision Learners

Masking Methos

- Similar to the original ViT, divide the input into non-overlapping patches.
- Uniformly sample a very large proportion (75%) of these patches and mask them
- Masking a high ratio makes predicting the task challenging and meaningful.
- Also, not using mask tokens and picking a high sampling ratio (masking most of the image, e.g., 75%, and sampling a small visible portion, e.g., 25%, to feed into the encoder) enables the encoder to be very large.

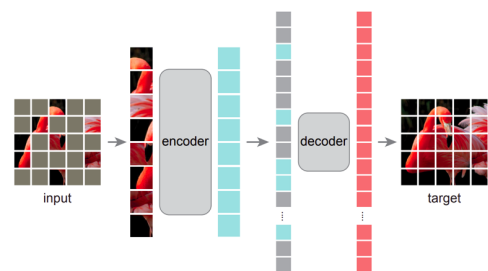


Figure 6. MAE architecture from the paper

He et al., 2021 Masked Autoencoders Are Scalable Vision Learners

MAE Encoder

- The encoder only operates on unmasked patches (25%)
- Embeds the patches by linear projection and add positional embeddings
- Uses transformer blocks
- Since the input patches are a small part of the input, the encoder is chosen to be very large. (encoder has over 9 times computations per token vs decoder)

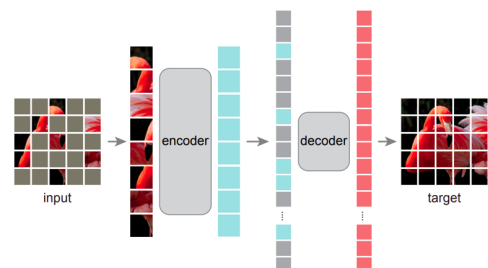


Figure 6. MAE architecture from the paper

He et al., 2021 Masked Autoencoders Are Scalable Vision Learners

MAE Decoder

- Merges the encoder outputs with the shared mask tokens in previously masked places, adding positional encodings to them.
- Uses transformer blocks, followed by a linear projection for finalizing pixel reconstruction.
- Is solely responsible for reconstruction, meaning it is not used post-training. Hence, it is independent of the encoder design, making it flexible (unlike traditional AEs or UNet). This is an **asymmetrical** autoencoder design.

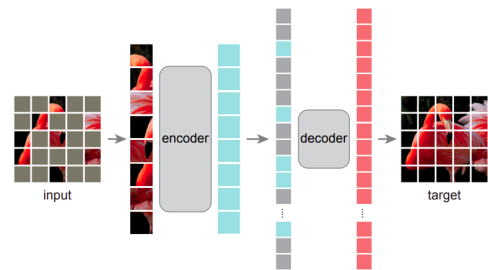


Figure 6. MAE architecture from the paper

He et al., 2021 Masked Autoencoders Are Scalable Vision Learners

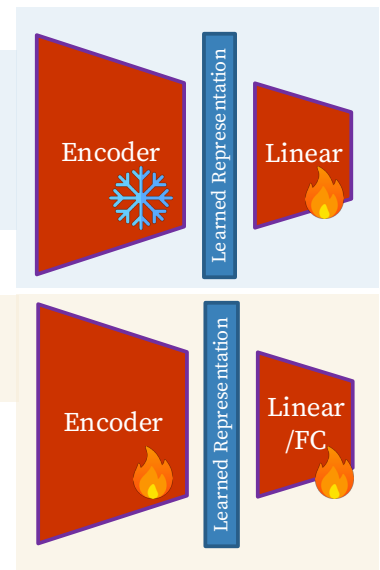
Reconstruction

- The MSE (mean squared error loss) in the pixel space between the input image and the reconstructed image is adopted.
- Loss is only computed for masked patches

He et al., 2021 Masked Autoencoders Are Scalable Vision Learners

Linear Probing vs Full Fine-tuning

- In linear probing, the pre-trained model is fixed, and only one linear layer is added at the end, to predict the labels (or produce the output). This method is used to assess the quality of representations from a pre-trained feature extraction model.
- In fine-tuning, pre-trained model is further trained (not fixed), and one or more layers, possibly with non-linearities are added.
- linear probing: provides a measure of representation quality of a pre-training in restricted conditions
- fine-tuning: exploits models near-true potential to adopt for new tasks



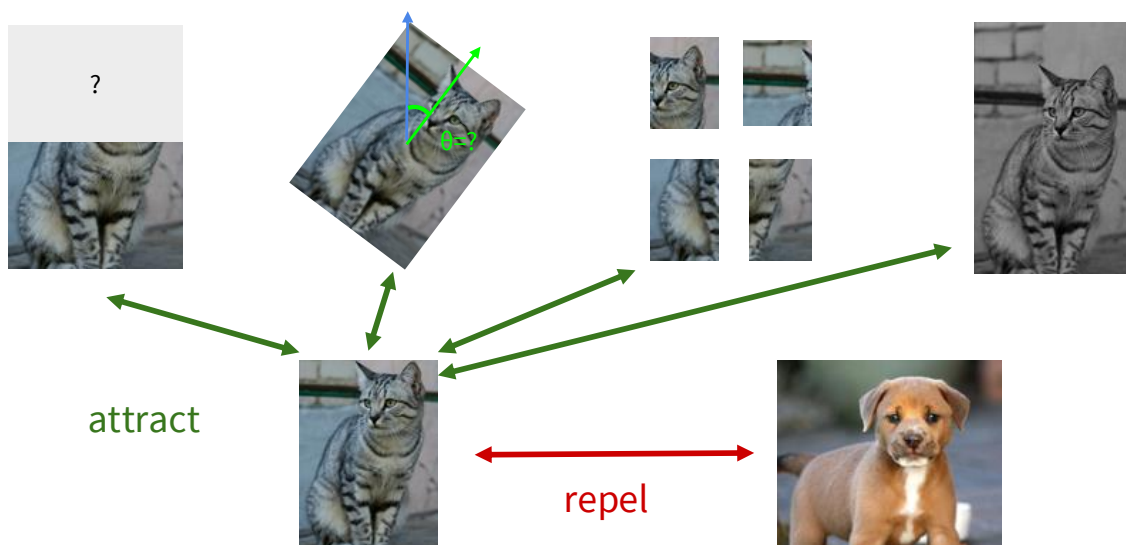
He et al., 2021 Masked Autoencoders Are Scalable Vision Learners

Summary: pretext tasks from image transformations

- Pretext tasks focus on “visual common sense”, e.g., predict rotations, inpainting, rearrangement, and colorization.
- The models are forced learn good features about natural images, e.g., semantic representation of an object category, in order to solve the pretext tasks.
- We often do not care about the performance of these pretext tasks, but rather how useful the learned features are for downstream tasks (classification, detection, segmentation).
- **Problems: 1) coming up with individual pretext tasks is tedious, and 2) the learned representations may not be general.**

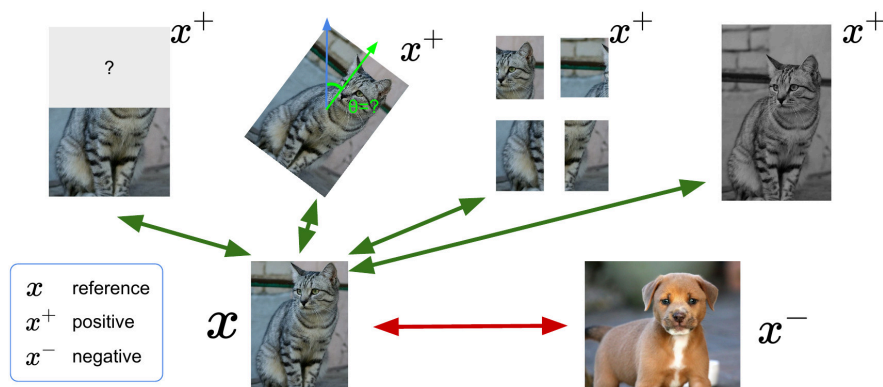
Contrastive Representation Learning

Contrastive Representation Learning



SimCLR

Contrastive Learning



- Given a chosen **score function** $s(\cdot, \cdot)$, we want to learn an encoder f that yields **high score for positive pairs** (x, x^+) and **low score for negative pairs** (x, x^-) :

$$s(f(x), f(x^+)) \gg s(f(x), f(x^-))$$

Contrastive Learning

Assume we have 1 reference (x), 1 positive (x^+) and $N-1$ negative (x_j^-) examples.
Consider the following **multi-class cross entropy loss function**:

$$\mathcal{L} = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

This is commonly known as the **InfoNCE loss** and its negative is a **lower bound** on the **mutual information** between $f(x)$ and $f(x^+)$ (See [Oord, 2018] for details):

$$MI[f(x), f(x^+)] \geq \log(N) - \mathcal{L}$$

Key idea: maximizing mutual information between features extracted from multiple “views” forces the features to capture information about higher-level factors.
Important remark: The larger the negative sample size (N), the tighter the bound.

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{score for the positive pair}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

Cross entropy loss for a N -way softmax classifier!

I.e., learn to find the positive sample from the N samples

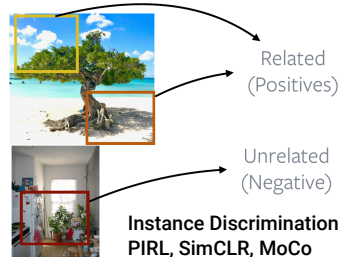
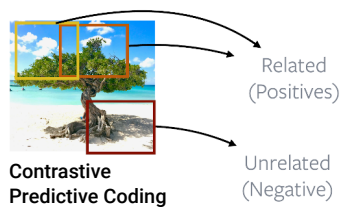
Design Choices

1. Score Function:

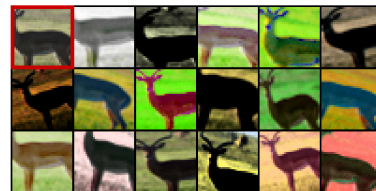
$$s(\mathbf{f}_1, \mathbf{f}_2) = \frac{\mathbf{f}_1^\top \mathbf{f}_2}{\|\mathbf{f}_1\| \|\mathbf{f}_2\|}$$

- Cosine similarity
- Commonly used

2. Examples:



3. Augmentations:



- Crop, resize, flip
- Rotation, cutout
- Color drop/jitter
- Gaussian noise/blur
- Sobel filter

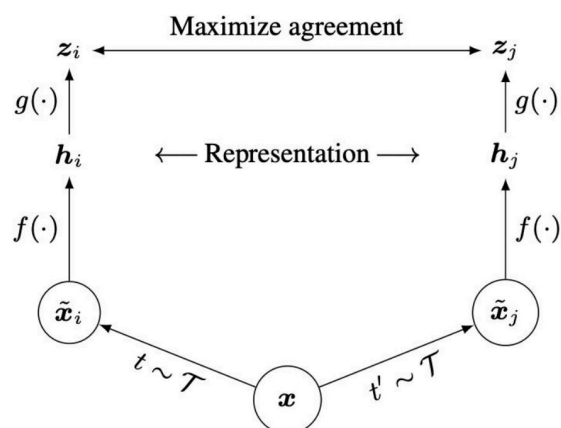
59

A Simple Framework for Contrastive Learning

- **Cosine similarity** as score function:

$$s(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\| \|\mathbf{z}_j\|}$$

- SimCLR uses a **projection network** $g(\cdot)$ to project features to a space where contrastive learning is applied
- The projection improves learning (more relevant information preserved in $\mathbf{h}$ which is discarded in $\mathbf{z}$)



A Simple Framework for Contrastive Learning

Algorithm 1 SimCLR's main learning algorithm.

```

input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do
  for all  $k \in \{1, \dots, N\}$  do
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$ 
    # the first augmentation
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$ 
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection
    # the second augmentation
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$ 
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection
  end for
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity
  end for
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$ 
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$ 
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$ 
end for
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 

```

Generate a positive pair by sampling data augmentation functions

Iterate through and use each of the 2N sample as reference, compute average loss

InfoNCE loss: Use all non-positive samples in the batch as $\mathbf{x}^-$

DINO

DINO: Self-Distillation with No Labels

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron^{1,2} Hugo Touvron^{1,3} Ishan Misra¹ Hervé Jegou¹
Julien Mairal² Piotr Bojanowski¹ Armand Joulin¹

¹ Facebook AI Research ² Inria* ³ Sorbonne University

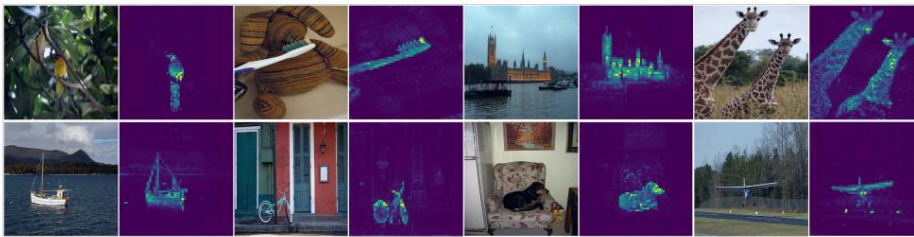
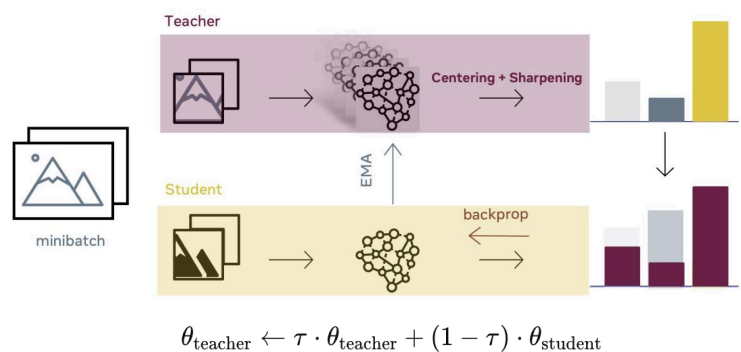
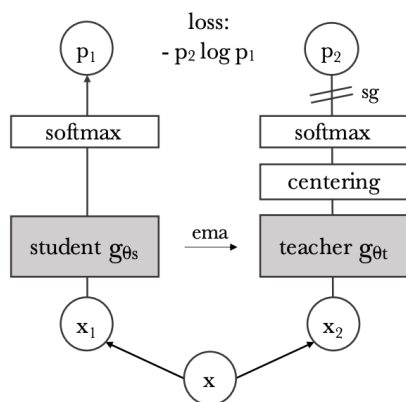


Figure 1: Self-attention from a Vision Transformer with 8×8 patches trained with no supervision. We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

Caron et al. 2021 Emerging Properties in Self-Supervised Vision Transformers

DINO



Caron et al. 2021 Emerging Properties in Self-Supervised Vision Transformers

DINO

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Caron et al. 2021 Emerging Properties in Self-Supervised Vision Transformers

DINO v2

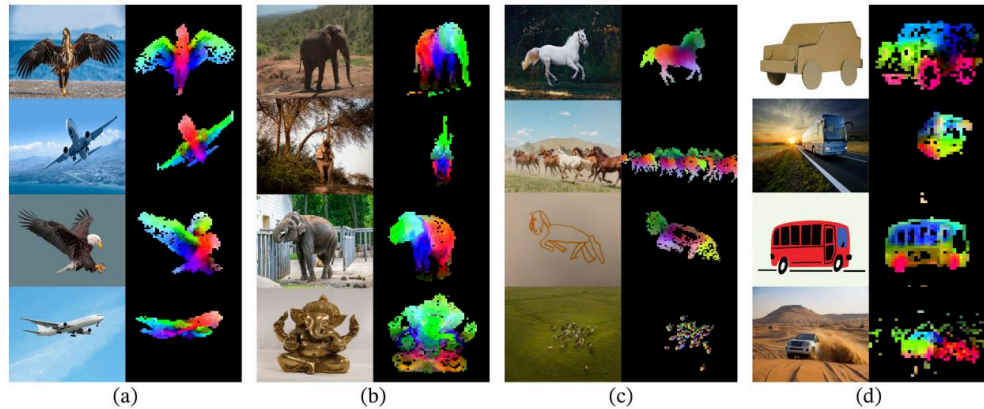


Figure 1: **Visualization of the first PCA components.** We compute a PCA between the patches of the images from the same column (a, b, c and d) and show their first 3 components. Each component is matched to a different color channel. Same parts are matched between related images despite changes of pose, style or even objects. Background is removed by thresholding the first PCA component.

Caron et al. 2021 Emerging Properties in Self-Supervised Vision Transformers